

07 — Quality & Self-Healing

Objetivo: garantizar que toda app generada por Puglit es *software que funciona* — que compila, que arranca, que sirve sus páginas sin un solo 5xx, que no expone secretos ni SQL injection, y que su lógica de dominio tiene tests reales con cobertura medida. No "código que parece correcto": código que se ejecuta y se demuestra ejecutando.

Esta sección documenta la capa de *quality gates* y *self-healing* del swarm: las que detectan los defectos que un modelo local inevitablemente produce, y las que los **reparan** en el mismo loop, sin intervención humana. Todo lo que sigue está verificado contra el código real en `/Users/alvaroz/projects/2026/puglit/web`.

Archivos de referencia:

Capa	Archivo	Rol
Static scans	<code>web/lib/swarm-checks.ts</code>	<code>securityScan</code> + <code>consistencyScan</code>
Auto-repair	<code>web/lib/swarm-repair.ts</code>	<code>repairPhantomTables</code> + <code>repairSecurityWithFrontier</code>
Adversarial review	<code>web/lib/adversarial-review.ts</code>	3 lentes + cross-examination + verdict
Frontier escalation	<code>web/lib/swarm-fitness.ts</code>	<code>frontierEscalate</code> (budget acotado)
CI loop (tsc real)	<code>web/lib/ci.ts</code>	dispatch GitHub Actions → parse tsc → Fixer
Runtime gate + QA	<code>web/scripts/build-local.mjs</code>	smoke test, SQL/tsc repair, Queen evidence review

1. Quality Philosophy

1.1 El problema: un compilado verde miente

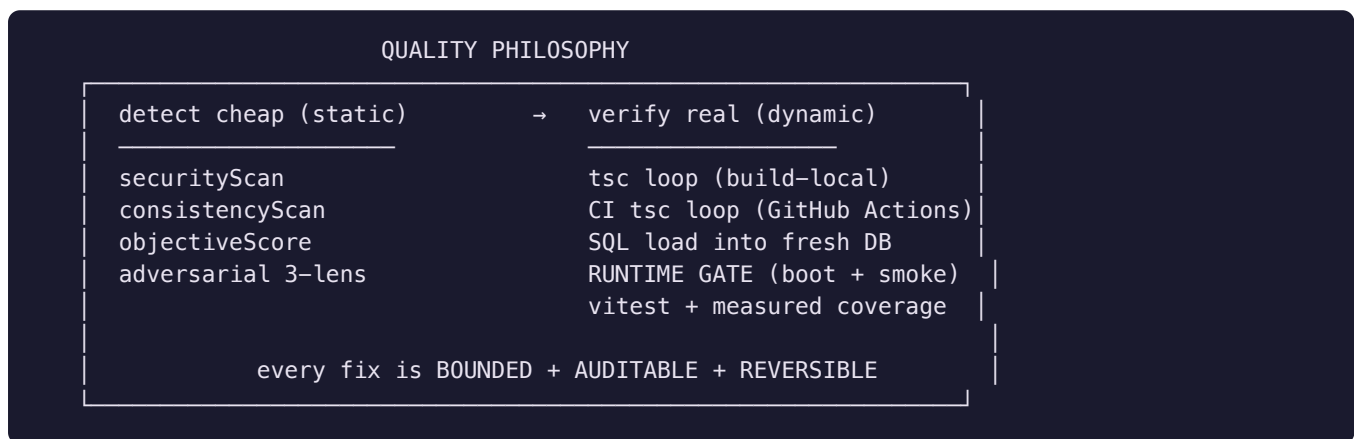
Los modelos chicos que corren 100% local (qwen2.5-coder, deepseek-coder, devstral en Ollama) producen código que a menudo **compila pero no corre**. Alucinan tablas que el schema nunca declara, escriben SQL por interpolación de strings, importan libs inexistentes, generan TSX sobre-escapado que tira 500 en cada página. Un `tsc --noEmit` verde no captura nada de esto.

La filosofía de Puglit es directa: **medir por ejecución, no por compilación**. Un check sólo cuenta si el artefacto pasa por una verificación que un humano repetiría a mano — arrancar la app y pegarle a las páginas, cargar el SQL en una base fresca, correr los tests y *medir* la cobertura. Las verificaciones estáticas (security, consistency) existen para acelerar la detección, pero el árbitro final es siempre dinámico.

Este principio se hace literal en el **runtime gate** (`build-local.mjs` , sección 2): después de que todo compila, se bootea la app con `next dev` , se camina el árbol `app/` , y se le pega un GET a cada página estática. Cualquier 5xx es un fallo del gate. La nota del proyecto sobre **Stayforge** (el clon de Airbnb usado como litmus test) lo resume: el runtime gate expuso **7 bugs reales invisibles a un compilado verde**.

1.2 Detección barata, reparación acotada

Toda reparación en Puglit es **bounded**: un presupuesto fijo de rondas, de archivos por ronda, de tokens por archivo. Nunca un loop abierto. El objetivo no es perseguir la perfección sino **subir el piso** — convertir "no arranca" en "arranca", "0% coverage" en "≥ el bar". Cuando una reparación no converge, el sistema preserva la mejor versión vista (anti-oscilación) y entrega con deuda registrada, no con un crash.



2. Build Validation

La validación de build tiene dos motores complementarios: el **loop local** (`build-local.mjs` , para preview/export sin GitHub) y el **loop CI** (`ci.ts` , contra GitHub Actions con un `tsc` real).

2.1 Runtime gate / smoke test

El corazón del principio "medir por ejecución" vive en `smokeTest()` de `build-local.mjs` . Después de servir la app con `next dev -p $PORT` :

1. **Espera el boot** — hasta 40 intentos de `fetch /` , esperando un status `< 500` . Si nunca responde, es `RUNTIME GATE FAIL` con razón `no-boot` .
2. **Descubre rutas estáticas** — camina recursivamente el árbol `app/` , recolectando cada `page.tsx` / `page.jsx` . Salta deliberadamente rutas dinámicas (`[...]`), grupos (`(...)`) y `api/` — sólo GETs sin efecto secundario.
3. **Pega a cada página** (hasta 20) — cualquier respuesta `>= 500` cuenta como fallo.
4. **Verdict** — `okGate = fail === 0` . Se escribe el resultado al status file (`pagesOk` , `pagesFail` , `failing`).

```

RUNTIME GATE (build-local.mjs smokeTest)
└─ next dev → wait boot (≤40× fetch /, status<500)
    │
    └─ no boot → x RUNTIME GATE FAIL (no-boot)
    ↓
    walk app/ → {/, /status, /incidents, ...} (skip [dyn], (group), api)
    ↓
    GET each (≤20) → any 5xx?
    └─ yes → x FAIL (failing: [/x→500, ...])
    └─ no → ✓ PASS (pagesOk: N)

```

Este gate es la diferencia entre "el código existe" y "el producto funciona". Captura `ReferenceErrors` por imports faltantes, TSX inválido, queries contra tablas inexistentes en runtime — toda la clase de bug que sobrevive a `tsc`.

2.2 tsc loop (local)

Antes del runtime gate, `repairTs()` garantiza que el proyecto compila. El loop:

- Corre `npx tsc --noEmit`, parsea errores por archivo.
- Para cada archivo con errores, le manda al modelo local los errores **reales** del compilador + el archivo, con un system prompt que prohíbe deps npm (sólo el spine: `@/lib/db`, `@/lib/auth`, etc.), exige `"use client"` cuando hay hooks, y rutas con `export async function GET/POST`.
- **Anti-oscilación:** guarda un snapshot de la *mejor* versión vista (menos errores) de cada archivo. Si 2 rondas no mejoran, restaura el mejor y para — nunca deja el proyecto peor de lo que lo encontró. `MAX_ROUNDS` default 6.

2.3 tsc loop (CI, real compiler)

`ci.ts` corre la verificación contra el **compilador real** de GitHub Actions, no una heurística:

Función	Qué hace
<code>dispatchCi(slug)</code>	Dispara el workflow <code>verify-generated.yml</code> sobre <code>main</code>
<code>latestRun / getRun</code>	Polling de status/conclusion del run
<code>runErrors(runId, slug)</code>	Baja los logs del job y parsea los errores <code>TS####</code> con regex, prefijando paths relativos a <code>projects/<slug>/</code>
<code>fixFiles(errors)</code>	El Fixer : por cada archivo (≤3), baja el contenido vía Contents API, lo repara contra los errores reales, lo re-pushea con <code>sha</code>

El system prompt del Fixer codifica la regla anti-deps: ante `Cannot find module 'X'`, **no** agrega el paquete — reescribe el uso al equivalente del spine (`jsonwebtoken / jose / bcrypt / axios` → `@/lib/auth`, `@/lib/db`, `@/lib/mailer`), y `router.query` → `useParams()`. Esto cierra el loop contra errores que son ground truth (los emite el compilador de CI), no contra una aproximación.

3. Security Validation

`securityScan(files)` en `swarm-checks.ts` corre sobre todo archivo `.ts/.tsx/.js/.mjs` generado. Está inspirado en NVIDIA/SkillSpector y emite `CodeIssue { severity, kind, file, detail }`. Cuatro clases:

Clase (<code>kind</code>)	Severidad	Patrón detectado
<code>hardcoded-secret</code>	high	<code>sk-...</code> , <code>AKIA...</code> , <code>ghp_...</code> — una clave/token real committeado
<code>hardcoded-credential</code>	med	<code>password\ secret\ api_key\ token = "literal"</code> sin <code>process.env</code>
<code>dangerous-exec</code> (RCE)	high	<code>eval(</code> , <code>new Function(</code> , <code>child_process</code> , <code>exec(</code>
<code>sql-injection</code>	high	<code>.query(\ ...\${...})</code> — SQL por interpolación, no <code>\$1,\$2`</code>
<code>xss</code>	med	<code>dangerouslySetInnerHTML</code> sin <code>sanitize / DOMPurify / escapeHtml</code>

La regex de `hardcoded-credential` tiene una sutileza importante: sólo dispara si el match **no** contiene `process.env`, para no marcar la lectura correcta de un secret desde el entorno. Es una detección estática, barata, que corre en finalize y cuyos findings van al build log y al critic para que el swarm se autocorrija; los **high** de seguridad además alimentan la frontier escalation (sección 11).

4. Consistency Validation

`consistencyScan(files, declaredTables)` ataca la clase de bug más recurrente del agente: el **phantom-table** — SQL que lee/escribe una tabla que el schema nunca declara. Es el "bug de las 5 tablas inventadas" / "FK-to-users". Inspirado en colbymchenry/codegraph.

El algoritmo:

1. **Construye el conjunto de tablas conocidas:** - `declaredTables` (del blueprint), - más toda tabla `CREATE TABLE` encontrada en cualquier `.sql`, - más las **SPINE_TABLES** que el spine siempre provee: `users`, `sessions`, `accounts`, `analytics_events`, `password_resets`, `magic_links`, `records`, - más el namespace Postgres (`information_schema`, `pg_*`, `public.`).
2. **Escanea cada `.ts/.tsx`** buscando `from|join|into|update <tabla>`. Si la tabla referenciada no es conocida (y tiene ≥ 3 chars, y no es keyword SQL), emite `phantom-table` **high**.
3. **Imports faltantes:** detecta `from "@/lib/<x>"` donde el archivo no existe y `<x>` no es una spine lib (`db`, `auth`, `auth-guards`, `mailer`, `rate-limit`, `analytics`, ...) → `missing-import` med.
4. **De-dup** por `kind+file+detail`.

```

consistencyScan - phantom-table detection
known = declaredTables u CREATE TABLE (*.sql) u SPINE_TABLES u pg_*

for each .ts/.tsx:
  match from|join|into|update T
  |
  | known(T)?  —yes→ ok
  |             —no→ Δ phantom-table (high)
  |                   "query references T never CREATED"

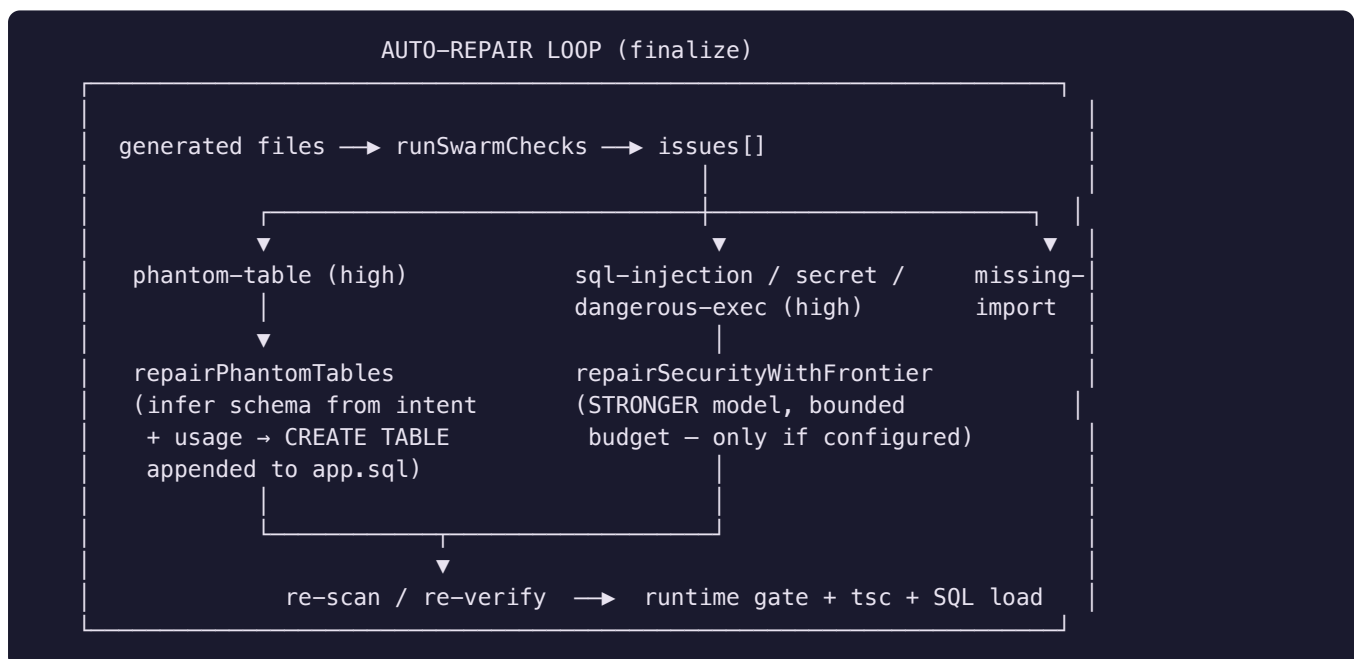
```

Hay un refuerzo *objetivo* del mismo defecto antes de generar: `objectiveScore` (en `swarm-fitness.ts`, usado por el torneo genético) **penaliza** blueprints cuyas rutas referencian tablas no declaradas (`-4` por phantom, hasta `-20`). Así la consistencia se vigila tanto en selección (blueprint) como en finalize (código).

`runSwarmChecks(files, declaredTables)` corre ambos scans y devuelve un resumen compacto (`Δ N` quality issues (H high): kind@file, ... o `✓ clean`).

5. Auto Repair Engine

Detectar no alcanza: una crítica explícita del diseño es que "auto-repair sólo flaggea security" y que "el modelo tiene un techo". El engine de reparación cierra el loop sobre las clases de mayor impacto. El flujo completo:



A esto se suma, en `build-local.mjs`, la reparación de **SQL** (carga en base fresca, sección 6 extendida), **imports** (fixer determinístico, sección 7), **schema** (DDL que el modelo erró, sección 8) y **tests** (QA con Queen review, sección 9). Las próximas secciones detallan cada motor.

6. Database Repair (`repairPhantomTables`)

Es la reparación de mayor impacto: convierte una alucinación de schema en un `CREATE TABLE` válido, así el build deja de shippear código que crashea en una relación inexistente. Vive en `swarm-repair.ts`.

Entradas: los `phantom-table` issues, los `declaredTables`, y opcionalmente el `intent` del arquitecto (tablas + columnas que el blueprint *quería*).

Algoritmo:

1. Extrae los nombres de las phantom tables de los issues.
2. Para cada una, recolecta hasta 8 **snippets de uso** (los `from/into/update/join <tabla>`) de los `.ts/.tsx` — así el modelo infiere columnas a partir de *cómo se usa* la tabla.
3. Arma el `intentText` con el schema **intencionado** del arquitecto como ground truth.
4. Una llamada `chatJSON` con `MODELS.premium`, `temperature 0.1`, schema `{ sql }`. El system prompt manda: **primero decidir si la phantom es un typo/alias de una tabla intencionada** (y en ese caso mapearla con un `CREATE VIEW alias AS SELECT * FROM real`, *no inventar*), y **sólo si es genuinamente nueva**, inferir columnas/tipos/keys del uso.
5. Si el output contiene `CREATE TABLE / CREATE VIEW`, lo **appendea a `sql/app.sql`** con un marcador de **backup comentado** del SQL pre-repair — la reparación es auditable y reversible.

Dos salvaguardas de diseño explícitas (anotadas en el código como *crítica*):

- **Anti-persistencia de alucinaciones:** prefiere la intención declarada del blueprint sobre la inferencia desde una query potencialmente errónea. Un typo se mapea, no se materializa como tabla nueva.
- **Reversibilidad:** el SQL original queda preservado y comentado encima del marcador, de modo que una reparación mala es visible y deshacible.

El loop de carga de SQL en `build-local.mjs` (`loadAppSchemaRepairing`) es la verificación *dinámica* que respalda a `repairPhantomTables`: por hasta 5 rondas resetea una base fresca, aplica las migraciones del spine, carga `app.sql` y `seed.sql` con `ON_ERROR_STOP=1`, y ante error le pasa el error de psql + el SQL al modelo para que lo corrija (orden de dependencias, nada de `ENUM` inline → `TEXT + CHECK`, tipos válidos, columnas/FKs a tablas inexistentes removidas). Detección estática + carga real = el schema se *demuestra* cargando.

7. Import Repair

Dos niveles, uno estático y uno determinístico:

Detección (estática) — `consistencyScan` emite `missing-import` para `@/lib/<x>` inexistente que no sea spine lib (sección 4).

Reparación (determinística) — `ensureSpineImports(code, rel)` en `build-local.mjs`. Los modelos chicos usan `pool`, `getAuthUser`, `NextResponse`, `NextRequest` **sin importarlos**, lo que produce `ReferenceErrors 500` que `tsc` puede no detectar. El fixer:

- Detecta el uso de cada símbolo con regex y verifica que no esté ya importado.

- Prepende el import correcto del spine: `pool` → `@/lib/db`, `getAuthUser` → `@/lib/auth`, `NextResponse / NextRequest` → `next/server`.
- Respeta la directiva `"use client"` líder, insertando los imports **después** de ella.

Además, el loop tsc de CI (`fixFiles` en `ci.ts`) repara imports a nivel compilador: ante `Cannot find module`, reescribe el uso al spine en lugar de agregar la dependencia. Es importante: las apps generadas tienen **cero deps npm externas** — todo cuelga del spine.

8. Schema Repair

Más allá de phantom tables, el modelo erra DDL que es sintácticamente plausible pero que Postgres rechaza. `loadAppSchemaRepairing` + el system prompt `SQL_FIX_SYS` lo cubren cargando el schema en una **base de verdad** y reparando contra el error real de psql:

- Definir cada tabla **antes** de referenciarla (orden de dependencias).
- **No** `ENUM(...)` inline en columna → `TEXT + CHECK`.
- Sin comas finales, sólo tipos Postgres válidos.
- Remover columnas/FKs que referencian tablas inexistentes.
- Mantener coherencia con el producto (para un status page: `components`, `statuses`, `incidents`, `uptime` — nada de dominios ajenos).

El seed se repara aparte, con la restricción extra de que los `INSERT` deben matchear nombres exactos de tabla/columna del schema. Si tras 5 rondas no converge, sigue: la app puede correr con tablas parciales, pero nunca se aborta el build por un schema imperfecto.

9. Test Repair (QA: vitest + coverage + Queen evidence review)

Esta es la encarnación más pura de "medir, no confiar en el claim". En vez de creer que el código está testeado, el gate **mide** y, si no alcanza, **rebota al equipo** con feedback específico.

El bloque `runTests()` de `build-local.mjs`:

1. **¿Hay tests?** Si no, la Reina exige cobertura del dominio y se genera una ronda inicial.
2. `runVitest()` — corre `npx vitest run --coverage --reporter=json`, parsea `numTotalTests/numPassedTests/numFailedTests` y la cobertura de líneas por archivo desde `coverage-summary.json`.
3. **Queen evidence review** — mientras haya tests rojos o cobertura `< COV_BAR` (default **70%**), y queden rondas (`QA_ROUNDS` default 2): - `uncovered(perFile)` identifica los **domain libs** (excluyendo `db/auth/mailer/rate/analytics/ i18n` y tests) cuya cobertura medida está bajo el bar. - La Reina los nombra explícitamente: *"X/Y verdes, coverage Z% < bar 70% — mostrame evidencia, falta cubrir: a.ts (41%), b.ts (12%)"*. Devuelto al equipo. - `writeTestsFor(rels)` genera tests vitest (Arrange-Act-Assert, behavior naming, sólo funciones puras — nunca DB/red, asserts exactos + casos borde) con anti-patterns prohibidos explícitamente (`expect(true).toBe(true)`, `snapshot-everything`, `test.skip`, debilitar asserts). - Re-mide.

4. **Veredicto:** `approved = failed === 0 && cov >= COV_BAR` . Se escribe al status (`tests, testsPassed, testsFailed, coverage, coverageBar, qaRounds, qaApproved`).



Lo decisivo: el bar es sobre **cobertura medida real**, no sobre una promesa del modelo. Si no se alcanza, no se bloquea ciegamente — se entrega con deuda de QA **registrada**, no oculta.

10. Retry Logic

Toda reparación es un loop acotado con anti-oscilación. Los presupuestos reales:

Loop	Archivo	Rondas	Anti-oscilación
tsc repair (local)	<code>build-local.mjs</code> <code>repairTs</code>	<code>MAX_ROUNDS</code> (6)	best-seen snapshot; para si 2 rondas sin mejora; restaura el mejor
SQL load repair	<code>build-local.mjs</code> <code>loadAppSchemaRepairing</code>	5	base fresca cada intento; sigue con tablas parciales si no converge
QA / coverage	<code>build-local.mjs</code> <code>runTests</code>	<code>QA_ROUNDS</code> (2)	re-mide cada ronda; entrega con deuda si no llega al bar
CI tsc Fixer	<code>ci.ts</code> <code>fixFiles</code>	≤3 archivos/pasada	reescribe contra error real; salta el archivo si falla
Adversarial repair	<code>adversarial-review.ts</code>	1 pasada bounded + re-review	≤3 archivos críticos; re-sintetiza desde review fresca

El patrón común: **nunca dejar el artefacto peor, siempre preservar la mejor versión, y parar temprano** cuando la mejora se estanca. Un fix que oscila se descarta a favor del snapshot con menos errores.

11. Escalation Logic (frontier escalation)

El modelo local tiene techo. Para los `high` de seguridad que la heurística/el modelo chico no pueden arreglar con seguridad, Puglit gasta un **presupuesto acotado de un modelo más fuerte**.

`frontierEscalate` vive en `swarm-fitness.ts`; `repairSecurityWithFrontier` lo usa en `swarm-repair.ts`.

`frontierEscalate(messages)` :

- No-op a menos que estén seteados `PUGLIT_FRONTIER_BUDGET` y `PUGLIT_FRONTIER_MODEL`.
- Lleva un contador `_frontierUsed`; `frontierRemaining() = budget - used`. Cuando se agota, no escala más — el costo queda **bounded por build**.
- Llama un endpoint OpenAI-compatible (`PUGLIT_FRONTIER_BASE_URL`, default `api.openai.com/v1`) con el modelo frontier configurado.

`repairSecurityWithFrontier(files, issues)` :

- Filtra los issues `high` de `sql-injection`, `hardcoded-secret`, `dangerous-exec`.
- Agrupa por archivo y, por cada uno, le pide al frontier que devuelva **el archivo completo corregido**: parametrizar todo el SQL (`$1, $2, ...`), mover secretos a `process.env`, remover `eval/exec/child_process`, manteniendo el comportamiento y que siga compilando bajo `tsc`.
- Limpia fences y reemplaza el contenido. Devuelve cuántos archivos arregló.

```
FRONTIER ESCALATION (bounded)
├── high security issue (sqli / secret / RCE)
│   │
│   ▼
│   PUGLIT_FRONTIER_BUDGET set? —no→ no-op (queda flaggeado en el log)
│   │ yes & remaining>0
│   ▼
│   STRONGER model rewrites the file (parameterize SQL, env secrets, drop eval)
│   │
│   ▼
│   _frontierUsed++ (cost capped per build)
```

La escalation es **selectiva por costo**: sólo la clase de defecto de mayor riesgo (seguridad high) y sólo cuando el operador habilitó un presupuesto. El 99% del trabajo lo hace el modelo local; el frontier es el escape hatch para lo que de verdad importa.

Adversarial Review (3 lentes + cross-examination + SHIP/CAVEATS/BLOCK)

`adversarial-review.ts` es la revisión pre-entrega: el patrón **addyosmani/adverse**, nativo de Puglit. **Tres lentes ortogonales** revisan el deliverable de forma independiente; luego una **cross-examination determinística** los confronta; el resultado es un verdict `SHIP / SHIP-WITH-CAVEATS / BLOCK` + una pasada de reparación acotada.

Las tres lentes

Una sola llamada de modelo por persona (`MODELS.premium` , `temp 0.2`), cada una con un mandato estricto de quedarse en su carril:

Lente	Dominio	Busca
Auditor	LOGIC & CORRECTNESS	math de dominio errado (pricing/totales/fechas/%), edge cases, off-by-one, mismatches de naming/schema (mismo concepto bajo dos nombres), magic values, reglas duplicadas
Adversary	ABUSE & SECURITY (sesgado a rechazar)	SQL injection, falta de auth/scoping por usuario (IDOR), race conditions, double-booking/double-spend, overflow, input no validado llegando a la DB, secretos
Pragmatist	DESIGN HEALTH & MAINTAINABILITY	lógica de negocio en route handlers en vez de <code>lib/</code> , violaciones DRY, rutas muertas, tests faltantes / cobertura baja del dominio, over-engineering

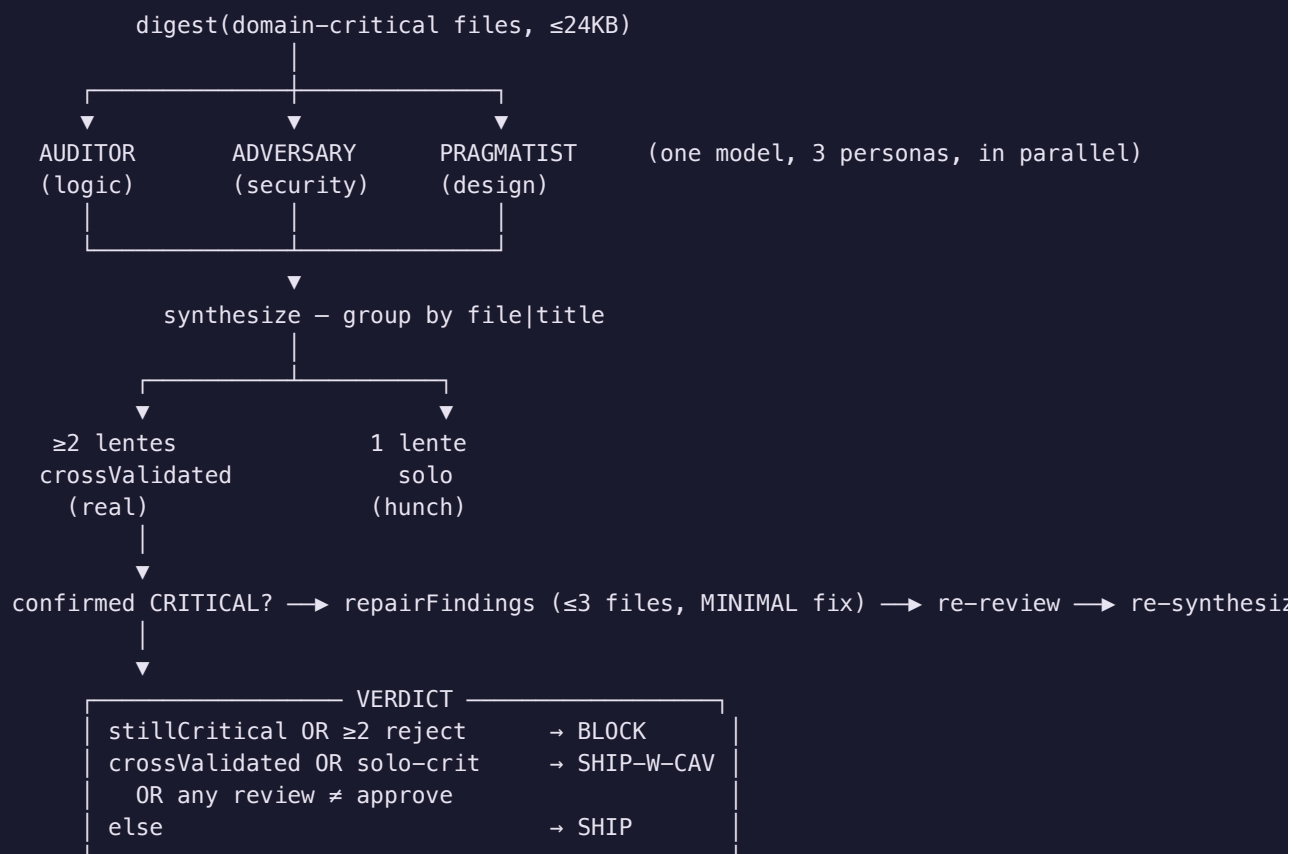
El diseño busca *consenso de lentes ortogonales* para cortar el **anchoring bias** de un solo modelo: una persona, tres posturas, más barato que multi-modelo. Cada lente devuelve `verdict` (`approve/conditional/reject`) + `findings[]` con severidad `critical/warning/info` y cita de archivo. Antes de pasarlo a las lentes, `digest(files)` arma un extracto enfocado de los archivos domain-critical (`lib/`, route handlers, `sql/app.sql`) acotado a ~24KB — los modelos locales tienen ventana chica.

Cross-examination

`synthesize(reviews)` agrupa findings por una clave normalizada (`archivo|título-normalizado`) y cuenta **cuántas lentes** reportaron cada uno:

- **≥2 lentes** → `crossValidated` — la señal real, confirmada (`[confirmado por auditor+adversary]`).
- **1 lente** → `solo` — una corazonada.

ADVERSARIAL REVIEW



Reparación + verdict

Si hay **CRITICAL cross-validado** (y no es `PUGLIT_TRAINING_MODE`), `repairFindings` corre una pasada **bounded**: hasta 3 archivos críticos, fix mínimo guiado por el finding (con el `PLAYBOOK.dev` inyectado), sin reescribir lo que funciona ni agregar deps. Luego re-digiere, re-revisa con las 3 lentes y re-sintetiza desde la review fresca.

El verdict final:

Condición	Verdict
Queda CRITICAL cross-validado o ≥ 2 lentes votaron <code>reject</code>	BLOCK
Hay findings cross-validados, o un solo-CRITICAL, o alguna lente no aprobó	SHIP-WITH-CAVEATS
Nada de lo anterior	SHIP

En **training mode** (batch de brain-fill) se revisa una vez y se saltea la cara pasada de repair + re-review. El resto del tiempo, "ojos externos antes de entregar": la cross-examination filtra ruido (descarta corazonadas de una sola lente) y el verdict gradúa la entrega entre limpio, con caveats, o bloqueado.

12. Success Metrics

Cómo se mide que la capa de quality funciona — no en abstracto, sino en señales que el sistema emite y persiste:

Métrica	Fuente	Significado
Runtime gate pass (<code>pages0k/pagesFail</code> , <code>smoke</code>)	<code>build-local.mjs smokeTest</code>	páginas estáticas servidas sin 5xx — el criterio "¿de verdad corre?"
tsc green	<code>repairTs</code> / CI run conclusion	compila sin errores tras el loop de reparación
SQL loads clean	<code>loadAppSchemaRepairing</code>	<code>app.sql</code> + <code>seed.sql</code> cargan en base fresca con <code>ON_ERROR_STOP</code>
Security findings → 0 high	<code>securityScan</code> + <code>repairSecurityWithFrontier</code>	secretos/SQLi/RCE detectados y reparados (o escalados)
Phantom tables reconciled	<code>repairPhantomTables</code> (return = # tablas)	alucinaciones de schema convertidas en DDL válido
Coverage ≥ bar (70%) + tests verdes	<code>runTests</code> (<code>qaApproved</code> , <code>coverage</code>)	lógica de dominio testeada con cobertura medida
Adversarial verdict	<code>adversarialReview</code>	SHIP / SHIP-WITH-CAVEATS / BLOCK con findings cross-validados

El proof point: Stayforge

El litmus test es **Stayforge**, un clon de Airbnb generado por el swarm: **80 archivos, 8 tablas**, anti-double-booking con `EXCLUDE USING gist` de Postgres, pricing determinístico en enteros-centavos, refund por snapshot de política, reviews doble-ciego. El dato que valida toda esta sección:

El runtime gate expuso 7 bugs reales — invisibles a un compilado verde.

Eso es exactamente la tesis de "medir por ejecución, no por compilación": siete defectos que sobrevivieron a `tsc` pero murieron al primer GET. Cada uno habría sido una app rota entregada al usuario; cada uno lo atrapó el smoke test del runtime gate.

El **batch de 100 builds de brain-training** (`infra/train-brain.sh` + `brain-training-ideas.tsv`) ejercita toda esta maquinaria a escala sobre apps diversas (juegos retro, calculadoras, trackers, verticales de negocio, dinero, catálogo, quiz), pre-cargando el brain con diary/exemplars/metrics/ skills — y cada build pasa por los mismos gates documentados arriba. La auto-healing no es un add-on: es el camino por defecto de toda app que Puglit entrega.